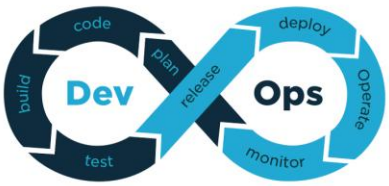
FATIMA JINNAH WOMEN UNIVERSITY
RAWALPINDI

Cloud DevOps



Lab 04

Simple Python Database Connections

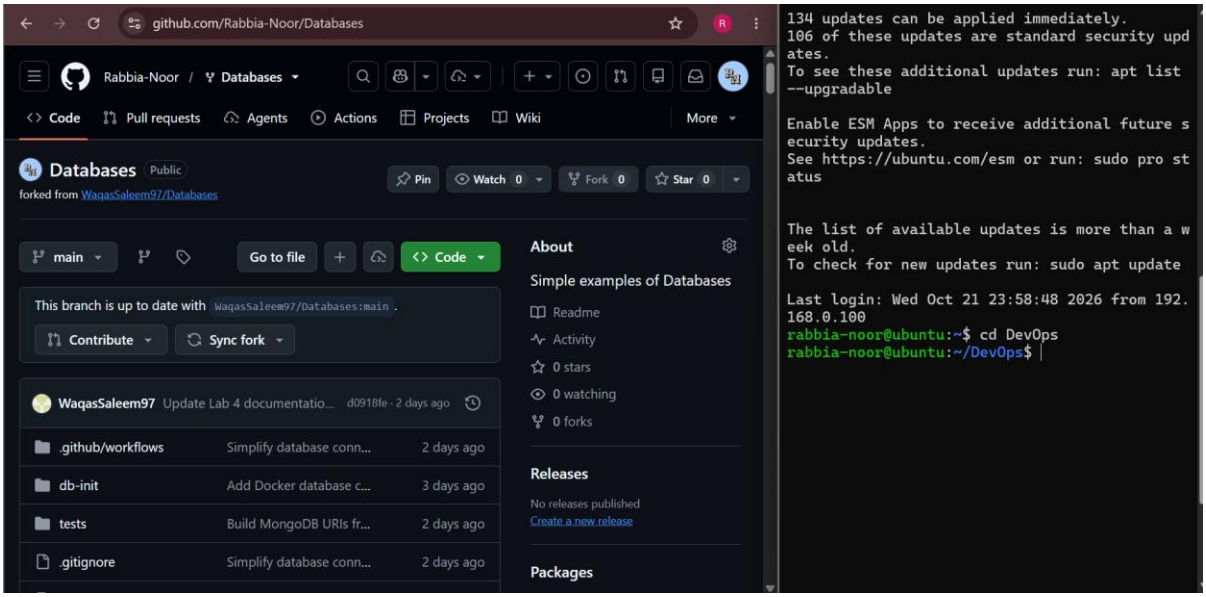
Submitted by: Rabbia Noor
Registration No.: 2023-BCS-070
Course Code: CS-649
GitHub Username: Rabbia-Noor
Section: A
Databases Fork: https://github.com/Rabbia-Noor/Databases.git

Table of Contents

Task 1 – Fork and Clone the Databases Repository 3
Task 2 – Install Docker, Docker Compose, and Python on Ubuntu 3
Task 3 – Start the Database Services 4
Task 4 – MySQL with Values in the Code 4
Task 5 – MongoDB with Values in the Code 5
Task 6 – MongoDB with os.getenv() 5
Task 7 – MongoDB with a Credentials File 6
Connection Results 7
Exam Evaluation Questions 8
Databases fork: 9

Task 1 – Fork and Clone the Databases Repository

Step 1: Forked the instructor's Databases repository into my GitHub account (Rabbia-Noor/Databases, forked from WaqasSaleem97/Databases). The terminal on the right shows the rabbia-noor@ubuntu prompt.



Step 2: Cloned my own fork on the Ubuntu server with git clone git@github.com:Rabbia-Noor/Databases.git, then moved into the Databases folder with cd Databases.

```
rabbia-noor@ubuntu:~$ git clone git@github.com:Rabbia-Noor/Databases.git
Cloning into 'Databases'...
remote: Enumerating objects: 101, done.
remote: Counting objects: 100% (101/101), done.
remote: Compressing objects: 100% (90/90), done.
remote: Total 101 (delta 30), reused 1 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (101/101), 39.86 KiB | 36.00 KiB/s, done.
Resolving deltas: 100% (30/30), done.
rabbia-noor@ubuntu:~$ cd Databases
rabbia-noor@ubuntu:~/Databases$
```

Task 2 – Install Docker, Docker Compose, and Python on Ubuntu

Step 1: After installing Docker and the Compose plug-in and reconnecting, git --version, docker --version and docker compose version all show their installed versions: Git 2.53.0, Docker 29.1.3 and Docker Compose v5.6.1.

```
rabbia-noor@ubuntu:~$ git --version
git version 2.53.0
rabbia-noor@ubuntu:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
rabbia-noor@ubuntu:~$ docker compose version
Docker Compose version v5.6.1
rabbia-noor@ubuntu:~$
```

Task 3 – Start the Database Services

Step 1: Ran docker compose up -d --wait mysql mongo-dev mongo-test mongo-prod, then docker compose ps. All four containers (MySQL and the three MongoDB databases) are up and healthy.

```
rabbia-noor@ubuntu:~/Databases$ docker compose up -d --wait mysql mongo-dev mongo-test mongo-prod
[+] up 9/9
 ✔Network databases_default         Created                                                    0.3s
 ✔Volume databases_mongo-dev-data   Created                                                    0.0s
 ✔Volume databases_mongo-prod-data  Created                                                    0.1s
 ✔Volume databases_mongo-test-data  Created                                                    0.1s
 ✔Volume databases_mysql-data       Created                                                    0.1s
 ✔Container databases-mysql-1       Healthy                                                   85.6s
 ✔Container databases-mongo-test-1  Healthy                                                   73.6s
 ✔Container databases-mongo-prod-1  Healthy                                                   67.6s
 ✔Container databases-mongo-dev-1   Healthy                                                   68.6s
rabbia-noor@ubuntu:~/Databases$ docker compose ps
NAME                    IMAGE       COMMAND                  SERVICE      CREATED              STATUS                          PORTS
databases-mongo-dev-1   mongo:7.0   "docker-entrypoint.s…"   mongo-dev    About a minute ago   Up About a minute (healthy)   127.0.0
.1:27017->27017/tcp
databases-mongo-prod-1  mongo:7.0   "docker-entrypoint.s…"   mongo-prod   About a minute ago   Up About a minute (healthy)   127.0.0
.1:27019->27017/tcp
databases-mongo-test-1  mongo:7.0   "docker-entrypoint.s…"   mongo-test   About a minute ago   Up About a minute (healthy)   127.0.0
.1:27018->27017/tcp
databases-mysql-1       mysql:8.4   "docker-entrypoint.s…"   mysql        About a minute ago   Up About a minute (healthy)   33060/t
cp, 127.0.0.1:3307->3306/tcp
rabbia-noor@ubuntu:~/Databases$
```

Step 2: Ran docker compose port for each service. MySQL is on 127.0.0.1:3307, MongoDB dev on 27017, MongoDB test on 27018 and MongoDB prod on 27019.

```
rabbia-noor@ubuntu:~/Databases$ docker compose port mysql 3306
127.0.0.1:3307
rabbia-noor@ubuntu:~/Databases$ docker compose port mongo-dev 27017
127.0.0.1:27017
rabbia-noor@ubuntu:~/Databases$ docker compose port mongo-test 27017
127.0.0.1:27018
rabbia-noor@ubuntu:~/Databases$ docker compose port mongo-prod 27017
127.0.0.1:27019
rabbia-noor@ubuntu:~/Databases$
```

Task 4 – MySQL with Values in the Code

Step 1: Ran python db_connect.py with the MySQL username, password, host, port and database written in the code. It connected to school_mysql on 127.0.0.1 port 3307, and the stored environment value is dev.

```
(.venv) rabbia-noor@ubuntu:~/Databases$
(.venv) rabbia-noor@ubuntu:~/Databases$ python db_connect.py
Connected to MySQL: school_mysql
Server: 127.0.0.1 3307
Environment: dev
(.venv) rabbia-noor@ubuntu:~/Databases$ s
```

Task 5 – MongoDB with Values in the Code

Step 1: Replaced db_connect.py with the MongoDB version that has the username, password, endpoint and database name written in the code. The ping succeeded and the output shows school_dev on port 27017 with environment dev.

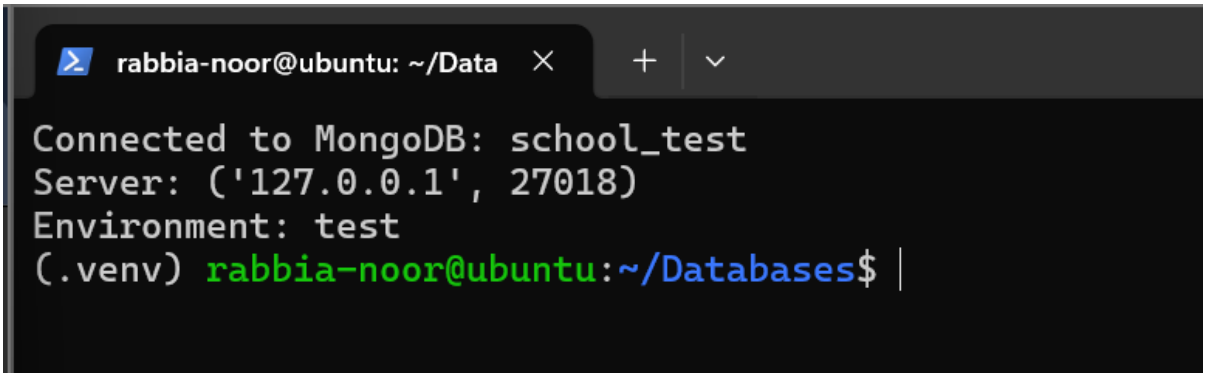
```
(.venv) rabbia-noor@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) rabbia-noor@ubuntu:~/Databases$
```

Task 6 – MongoDB with os.getenv()

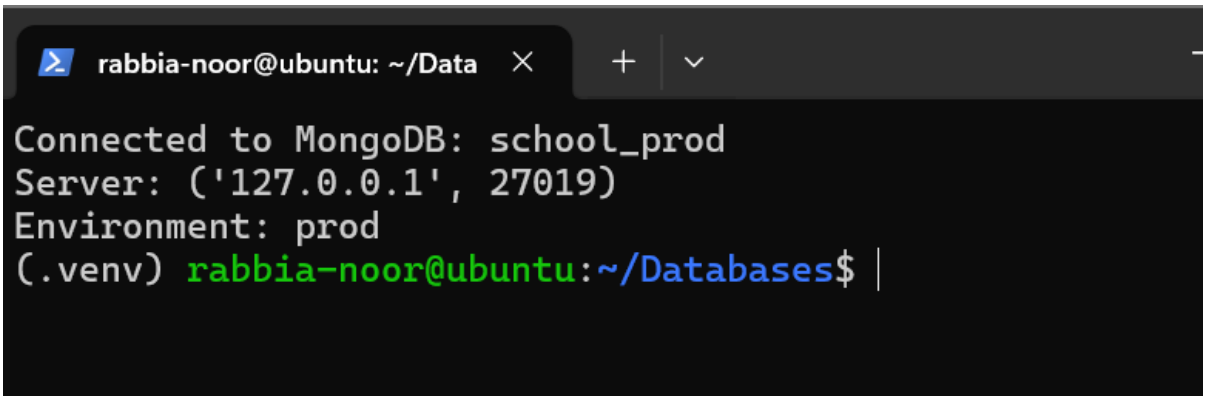
Step 1: Exported the four dev variables (MONGO_USERNAME, MONGO_PASSWORD, MONGO_ENDPOINT, MONGO_DATABASE), cleared the screen and ran the unchanged program. It connected to school_dev on port 27017, environment dev.

```
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) rabbia-noor@ubuntu:~/Databases$
```

Step 2: Exported the four test values and ran the same program. It connected to school_test on port 27018, environment test.

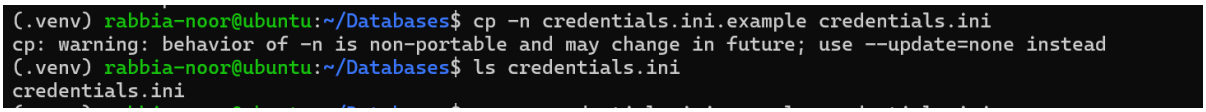


Step 3: Exported the four simulated prod values and ran the same program. It connected to school_prod on port 27019, environment prod.

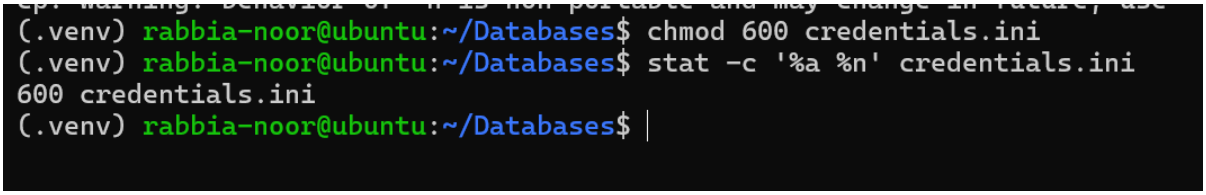


Task 7 – MongoDB with a Credentials File

Step 1: Created the local credentials file by copying credentials.ini.example to credentials.ini with cp -n, and confirmed it exists with ls credentials.ini.



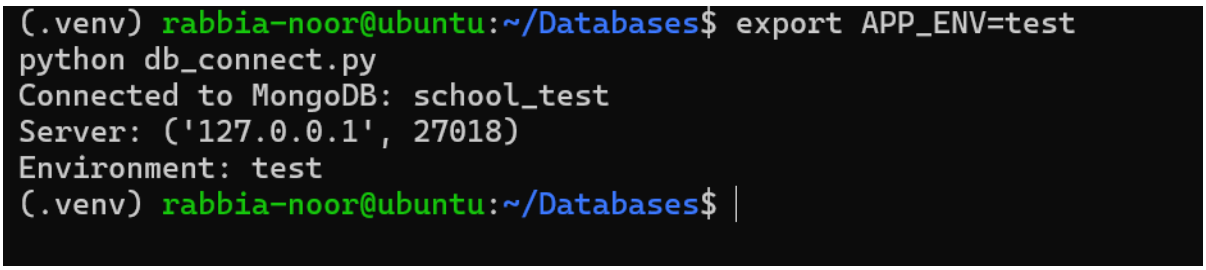
Step 2: Restricted the file to my account with chmod 600 credentials.ini. stat -c '%a %n' credentials.ini shows 600 credentials.ini.



Step 3: Switched db_connect.py to read credentials.ini with configparser. With APP_ENV=dev it connected to school_dev on port 27017, environment dev.



Step 4: With APP_ENV=test the same code read the [test] section and connected to school_test on port 27018, environment test.



Step 5: With APP_ENV=prod the same code read the [prod] section and connected to school_prod on port 27019, environment prod.

```
(.venv) rabbia-noor@ubuntu:~/Databases$ export APP_ENV=prod
python db_connect.py
Connected to MongoDB: school_prod
Server: ('127.0.0.1', 27019)
Environment: prod
(.venv) rabbia-noor@ubuntu:~/Databases$
```

Connection Results

Result of each connection, confirmed by the database name, port and the environment value stored in the database.

Target	Endpoint	Database	Environment
MySQL (in code)	127.0.0.1:3307	school_mysql	dev
MongoDB (in code)	127.0.0.1:27017	school_dev	dev
MongoDB (os.getenv) dev	127.0.0.1:27017	school_dev	dev
MongoDB (os.getenv) test	127.0.0.1:27018	school_test	test
MongoDB (os.getenv) prod	127.0.0.1:27019	school_prod	prod
MongoDB (file) dev	127.0.0.1:27017	school_dev	dev
MongoDB (file) test	127.0.0.1:27018	school_test	test
MongoDB (file) prod	127.0.0.1:27019	school_prod	prod

Exam Evaluation Questions

Exam Question 1 – A Missing Environment Variable

Result: With the prod values exported, I ran unset MONGO_DATABASE and then python db_connect.py. The program stopped with: Set all four MongoDB environment variables first.

```
(.venv) rabbia-noor@ubuntu:~/Databases$ nano db_connect.py
(.venv) rabbia-noor@ubuntu:~/Databases$ export MONGO_USERNAME=prod_user
export MONGO_PASSWORD='ProdDemoOnly123!'
export MONGO_ENDPOINT=127.0.0.1:27019
export MONGO_DATABASE=school_prod
unset MONGO_DATABASE
python db_connect.py
Set all four MongoDB environment variables first.
(.venv) rabbia-noor@ubuntu:~/Databases$
```

Explanation: The call os.getenv("MONGO_DATABASE") returned None because the variable no longer existed. The if statement checks "not database_name", which is true for None, so the program exits with SystemExit before it creates a connection. Exporting MONGO_DATABASE=school_prod again lets the program connect normally.

```
Environment: prod
(.venv) rabbia-noor@ubuntu:~/Databases$ nano db_connect.py
(.venv) rabbia-noor@ubuntu:~/Databases$ export MONGO_USERNAME=prod_user
export MONGO_PASSWORD='ProdDemoOnly123!'
export MONGO_ENDPOINT=127.0.0.1:27019
export MONGO_DATABASE=school_prod
unset MONGO_DATABASE
python db_connect.py
Set all four MongoDB environment variables first.
(.venv) rabbia-noor@ubuntu:~/Databases$ export MONGO_DATABASE=school_prod
python db_connect.py
Connected to MongoDB: school_prod
Server: ('127.0.0.1', 27019)
Environment: prod
(.venv) rabbia-noor@ubuntu:~/Databases$
```

Exam Question 2 – Which Source Does the File Version Read?

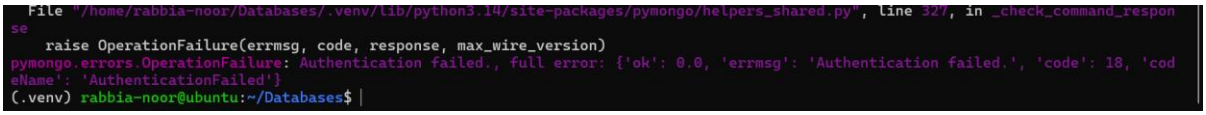
Result: The prod username, password, endpoint and database variables were still exported. After unset APP_ENV, the program connected to school_dev (port 27017, environment dev).

```
(.venv) rabbia-noor@ubuntu:~/Databases$ export MONGO_USERNAME=prod_user
export MONGO_PASSWORD='ProdDemoOnly123!'
export MONGO_ENDPOINT=127.0.0.1:27019
export MONGO_DATABASE=school_prod
unset APP_ENV
python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) rabbia-noor@ubuntu:~/Databases$
```

Explanation: The credentials-file version never reads the MONGO_* variables. It only reads APP_ENV to choose a section in credentials.ini, and os.getenv("APP_ENV", "dev") falls back to dev when APP_ENV is not set. So it loaded the [dev] section and ignored the prod values still in the shell.

Exam Question 3 – A Wrong Password in the File

Result: In the [test] section of credentials.ini I temporarily set password = wrong, then ran export APP_ENV=test and python db_connect.py. MongoDB returned: Authentication failed (code 18, AuthenticationFailed).



Explanation: The endpoint only tells Python where the server is. The server still checks the username and password before it allows any command, so the correct endpoint cannot make a wrong password valid. After restoring the supplied test password the connection works again.

Databases fork:

https://github.com/Rabbia-Noor/Databases